

An Exact Additive Decomposition of the MARINA cls Token

Attributing the cross-attention read budget to input modalities

Anthony Tong

30 July 2026

Abstract

MARINA’s cross-attention stack reads from a memory that is constructed once and never updated across blocks. This note shows that this single architectural property makes the final CLS token decompose *exactly* into a sum of per-source terms, and derives that decomposition in full: the span partition, the source-wise split of multi-head attention, the linear transport of an additive component through LayerNorm, and the resulting closed form for the output. The decomposition induces a signed budget over sources that sums identically to one — the *attribution*, or “usage”, measure. §10–§14 report the measured budget on three trained seeds ($n = 23,648$ molecules each), set it against the *ablation*, or “irreplaceability”, measure obtained by withholding each modality, refine it by depth into per-block *injection* and *survival*, and close with an attention-suppression experiment. The final section states precisely where exactness ends — the feedforward nonlinearity and the input-dependence of the attention weights — and why that causal instrument is required at all. Implementation references are to `src/modules/marina/model.py` on branch `modality-attribution`; §2 separates what is pre-existing trained-model code from what is measurement apparatus added for this analysis.

Contents

1	Vocabulary	3
2	What is pre-existing and what is instrumentation	3
3	Notation	4
4	The static-memory property	5
5	The span partition	5
5.1	The bucket inventory	5
5.2	What the m :token bucket is	6
6	Attention splits exactly by source	7
6.1	What <code>nn.MultiheadAttention</code> computes	7
6.2	How the instrumentation splits it	7
7	LayerNorm is linear in the components of its input	8
8	The block recursion and transport operators	9
9	From vectors to a budget that sums to one	10
10	Results: the modality budget	11
11	Results: redundancy within the NMR block	13
12	The other redundancy: the fingerprint target	14
12.1	The target carries far less information than its width	14
12.2	The redundancy buys nothing	15
12.3	What this means for the numbers above	16
13	Results: refining the budget by depth	17
14	Results: attention-depth suppression	19
15	Where exactness ends	20

1 Vocabulary

Six terms are used throughout in a specific sense. Three name things that exist in the trained model; three name things the measurement introduces.

In the model.

Memory The matrix M that the global CLS query reads from: every modality’s encoded tokens, concatenated into one sequence. Built once per forward pass by `_encode` and never modified afterwards.

Segment The contiguous block of memory rows belonging to one modality. Modality m ’s segment has L_m rows: one mod-token followed by $L_m - 1$ peak rows. `_encode` returns the layout as `segments = [(m, L_m), ...]`.

Mod-token A learned per-modality vector τ_m , an `nn.Parameter` held in `self.mod_tokens`, prepended as the *first* row of modality m ’s segment. It is a “this modality is present” marker, playing the role a segment embedding plays in BERT. It is part of the trained model, not of this analysis. See §5.2.

In the measurement.

Span An index set $\mathcal{S}_k$ over memory rows, used to group rows that should be credited together. Spans exist only in the instrumentation; the model has no notion of them.

Bucket A named additive term in the decomposition of the final CLS. Each span yields one bucket, and there are three further buckets for things no span can own (`ffn`, `bias`, `cls_init`). Table 2 is the full inventory.

Contribution The vector $c_k \in \mathbb{R}^d$ that bucket k adds to the final CLS. Its scalar *share* is the projection (4) of that vector onto the output direction.

Four measures are then built on these, and are easy to conflate: *attribution* (usage — share of what is read, everything present), *ablation* (irreplaceability — cosine lost when a modality is withheld), *injection* (the norm of a contribution where it was written), and *survival* (how much of that contribution reaches the output). The first two are compared in §10, the second two defined in §13.

2 What is pre-existing and what is instrumentation

Three tiers of code appear in this note, and it matters which is which: only the first two ran during training and produced the weights being analysed. *Nothing in tier 3 participates in training, inference, or the loss.* It is read-only apparatus, added on branch `modality-attribution` in a commit that is **+175/−4** lines on `model.py` — and the four deleted lines are only `forward`’s encoder body being extracted verbatim into `_encode` so the instrumentation can reuse it.

The distinction is not bookkeeping pedantry. The equations in §6 and §7 describe what *stock PyTorch* computes; the propositions about them are claims that the instrumentation reproduces that computation exactly, and each is checkable against the stock layer’s own output. Where a code listing follows, its first line states its tier.

Table 1: Provenance. Tier 3 is measurement apparatus; it changes no weights and alters no forward pass.

Tier	Symbol	What
1. Stock PyTorch	<code>nn.MultiheadAttention</code>	the cross-attention layer itself; defined in §6
	<code>nn.LayerNorm</code>	defined in §7
	<code>nn.TransformerEncoder</code>	the per-modality self-attention in <code>_encode</code>
2. MARINA, pre-existing	<code>CrossAttentionBlock.forward</code>	the trained path: one block of attention $\rightarrow$ LN $\rightarrow$ FFN $\rightarrow$ LN
	<code>MARINA.forward</code>	the full stack; what training and evaluation call
	<code>_encode</code>	builds M ; extracted from <code>forward</code> , behaviour unchanged
3. Instrumentation (new)	<code>attn_contributions</code>	splits one block’s attention output by span (Prop. 2)
	<code>forward_contributions</code>	the full decomposition (Thm. 1)
	<code>_ln_component, _ln_sigma</code>	transports one component through a LayerNorm (Lem. 1)

3 Notation

Everything below is written *per sample*; the batch axis is suppressed, since no operation in the stack mixes across it. Vectors are column vectors in $\mathbb{R}^d$.

Symbol	Meaning	Code
d	model width	<code>args.dim_model</code>
H	attention heads; $d_h = d/H$	<code>args.heads</code>
N	cross-attention blocks ($N = 16$)	<code>len(self.cross_blocks)</code>
$\ell \in \{0, \dots, N - 1\}$	block index	<code>li</code>
$\mathcal{M}$	modality set, $ \mathcal{M} = 5$	<code>args.input_types</code>
L_m	segment length of modality m (peaks + mod-token)	<code>L + 1</code>
L	total memory length, $L = \sum_m L_m$	<code>joint_seq.size(1)</code>
$M \in \mathbb{R}^{L \times d}$	joint memory, rows M_j	<code>joint_seq</code>
$\pi \in \{0, 1\}^L$	key-padding mask	<code>joint_mask</code>
$q_\ell \in \mathbb{R}^d$	CLS state entering block ℓ	<code>global_token</code>
$g \in \mathbb{R}^d$	learned initial CLS query	<code>self.global_cls</code>
$\tau_m \in \mathbb{R}^d$	learned mod-token of modality m	<code>self.mod_tokens[m]</code>
$W_Q, W_K, W_V \in \mathbb{R}^{d \times d}, b_V$	attention in-projections	<code>attn.in_proj_*</code>
$W_O \in \mathbb{R}^{d \times d}, b_O$	attention out-projection	<code>attn.out_proj</code>
$\gamma_\ell^{(i)}, \beta_\ell^{(i)} \in \mathbb{R}^d$	LayerNorm gain / bias, $i \in \{1, 2\}$	<code>norm1, norm2</code>
ε	LayerNorm epsilon	<code>norm.eps</code>

We use γ, β for the LayerNorm parameters, following Ba et al. (2016), and reserve α for attention weights. The frozen-scale linearisation of LayerNorm in Section 7 is the standard device from

Elhage et al., *A Mathematical Framework for Transformer Circuits* (2021); there it is used to fold LayerNorm into adjacent linear maps, whereas here it is used to push a sum through it componentwise.

4 The static-memory property

This is the structural precondition for everything that follows.

```
# --- Tier 3: forward_contributions. The tier-2 forward() has the same shape:
# one _encode, then a loop over cross_blocks reusing the same joint_seq.
joint_seq, joint_mask, segments = self._encode(batch) # model.py:296 -- once
for li, block in enumerate(self.cross_blocks):
    parts, attn_out = block.attn_contributions(
        global_token, joint_seq, joint_mask, spans)
```

The memory M is built once and passed unchanged to all N blocks; only q_ℓ evolves. There is *no* self-attention over M between blocks, so a memory row never mixes with rows outside its own segment during the cross-attention stack.

Remark (What is and is not pure). Inside `_encode`, each modality *does* receive a self-attention stack — but a *separate one per modality*, applied before concatenation (`model.py:239`). Consequently row M_j in segment m is a mixture of modality m ’s own peaks and τ_m , and of nothing else. Purity therefore holds at the *modality* level, which is the level at which the decomposition makes claims. It does *not* hold at the level of individual peaks: one cannot attribute the final CLS to peak #7 of the HSQC spectrum by this construction.

In a standard encoder–decoder or decoder-only stack the construction fails outright, because the memory (respectively, the residual stream) mixes sources at every layer.

5 The span partition

Let o_m denote the offset of segment m in the concatenation. Define index sets $\mathcal{S}_k \subseteq \{1, \dots, L\}$ (`model.py:299–308`):

$$\mathcal{S}_{m:\text{token}} = \{o_m\}, \quad \mathcal{S}_m = \{o_m + 1, \dots, o_m + L_m - 1\}.$$

Proposition 1 (Completeness). *The family $\{\mathcal{S}_k\}_{k \in \mathcal{S}}$, where $\mathcal{S} = \mathcal{M} \cup \{m:\text{token}\}_{m \in \mathcal{M}}$, is a partition of $\{1, \dots, L\}$: pairwise disjoint and exhaustive.*

This is what makes the split of Section 6 exact rather than approximate — no position is double-counted and none is dropped. For five modalities, $|\mathcal{S}| = 10$ spans, hence ten source buckets.

5.1 The bucket inventory

Only the first row is “information read off the spectra”. The naming in the results tables follows this: Table 3’s *Share* column is the m bucket alone, its *Mod-token* column is the $m:\text{token}$ bucket, and *Peaks-only* renormalises the five m buckets to sum to one.

Table 2: The thirteen aggregate buckets, with their seed-1 shares. Every one is a vector $c_k \in \mathbb{R}^d$; the share is its projection (4). They sum to 1 by Corollary 1. (Theorem 1 indexes each source bucket by block as well, giving 178 before summing over ℓ .)

Bucket	Count	What the CLS read it from	Share
m	5	modality m ’s <i>peak</i> rows — the actual spectrum	0.2324
m :token	5	modality m ’s <i>mod-token</i> row alone	0.0071
ffn	1	nothing — written by the feedforward path	0.7199
bias	1	nothing — $b_O, \beta^{(1)}, \beta^{(2)}$	0.0406
cls_init	1	nothing — the learned initial query g	≈ 0
	13		1.0000

5.2 What the m :token bucket is

This bucket is the one most easily misread, so it is worth being explicit.

Each modality’s segment is $[\tau_m, \text{peak}_1, \dots, \text{peak}_{L_m-1}]$ — the learned mod-token first, then the encoded peaks. The instrumentation cuts each segment into *two* spans rather than one: the single row $\{o_m\}$ holding τ_m , and the $L_m - 1$ rows holding the peaks. So for HSQC there are two buckets: `hsqc`, what the CLS read off actual HSQC cross-peaks, and `hsqc:token`, what it read off the marker row.

Why split them. When a modality is withheld, every one of its peak rows is masked (`mask = (x.abs().sum(-1) == 0)`, `model.py:234`), so $\alpha_j = 0$ there and the m bucket is *exactly* zero. The m :token bucket is not: the marker row is never masked. Keeping them apart therefore separates *reliance on the spectrum* from *reliance on knowing the spectrum was supplied*. Collapsed into one bucket, a modality that is merely announced would look partly used.

What it is worth. Together the five mod-token buckets take 0.0071 of the budget on seed 1 — under 1%, and about 3% of what the peak buckets take. Individually they run from -0.0004 (`mass_spec:token`, which pulls slightly *against* the output direction) to $+0.0027$ (`hsqc:token`). The exception is `mw`, whose mod-token share ($+0.0021$) equals its peak share ($+0.0021$) — half of what molecular weight contributes is the bare fact that it was provided.

Remark (The bucket is not a clean “presence prior” in general). τ_m is prepended *before* modality m ’s self-attention stack runs (`model.py:237-239`), so when peaks are present the marker row absorbs information from them. The m :token bucket is then the mod-token position *after* that mixing, not the learned parameter in isolation. It is a clean presence prior only in the absent case, where the peak rows are all masked and the marker attends to itself alone. Read the modality-present mod-token shares as “what the CLS took from the segment’s summary row”, not as “the value of the prior”.

6 Attention splits exactly by source

6.1 What `nn.MultiheadAttention` computes

Tier 1 — stock PyTorch. Everything in this subsection is the standard layer’s own definition, not MARINA code and not instrumentation. Each `CrossAttentionBlock` owns one such layer, constructed `nn.MultiheadAttention(embed_dim=d, num_heads=H, bias=True, batch_first=True)`.

The layer holds its three input projections *packed* into one $3d \times d$ parameter `in_proj_weight` (rows $0:d$ for W_Q , $d:2d$ for W_K , $2d:3d$ for W_V) with `in_proj_bias` packed the same way, plus a separate `out_proj` carrying W_O and b_O . Given a query and a key/value sequence it projects both, splits the width into H heads of size $d_h = d/H$, forms scaled dot-product attention per head with masked positions driven to $-\infty$ before the softmax, takes the value-weighted sum, concatenates the heads and applies `out_proj`. Dropout is applied to the attention weights in training only; here the model is in `eval()` and `forward_contributions` is `@torch.no_grad()`, so it is inactive.

MARINA’s trained path calls this layer once per block and keeps only its output:

```
# --- Tier 2: MARINA, pre-existing. This is the path that trained the weights.
attn_out, _ = self.attn(query, key, value,
                        key_padding_mask=key_padding_mask,
                        need_weights=False) # <- weights discarded
q1 = self.norm1(query + attn_out)
out = self.norm2(q1 + self.ff(q1))
```

Written out, with a single query (the CLS), the layer computes the following. Write $\alpha^{(h,\ell)} \in \mathbb{R}^L$ for the attention row of head h at block ℓ :

$$\alpha_j^{(h,\ell)} = \frac{\exp(\langle W_Q^{(h)} q_\ell, W_K^{(h)} M_j \rangle / \sqrt{d_h}) \mathbf{1}[\pi_j = 0]}{\sum_{j'} \exp(\langle W_Q^{(h)} q_\ell, W_K^{(h)} M_{j'} \rangle / \sqrt{d_h}) \mathbf{1}[\pi_{j'} = 0]}, \quad \sum_{j=1}^L \alpha_j^{(h,\ell)} = 1,$$

with values $v_j^{(h)} = W_V^{(h)} M_j + b_V^{(h)}$. The block’s attention output is

$$a_\ell = \sum_{h=1}^H \sum_{j=1}^L \alpha_j^{(h,\ell)} W_O^{(h)} v_j^{(h)} + b_O. \quad (1)$$

Equation (1) is a restatement of the stock layer; nothing has been added yet. The key structural fact it exposes — the one the next subsection exploits — is that the output is a **sum over source positions j** .

6.2 How the instrumentation splits it

Tier 3 — new. `attn_contributions` is a second, read-only entry point on `CrossAttentionBlock`. It does not replace `CrossAttentionBlock.forward` and is never called by it; the trained path above is untouched.

Proposition 2 (Exact source-wise split). *Define, for each bucket $k \in \mathcal{S}$,*

$$p_{k,\ell} = \sum_{h=1}^H \sum_{j \in \mathcal{S}_k} \alpha_j^{(h,\ell)} W_O^{(h)} v_j^{(h)} \quad (2)$$

Then $\sum_{k \in \mathcal{S}} p_{k,\ell} + b_O = a_\ell$ identically.

Proof. The outer sum over j in (1) is a plain sum, and by Proposition 1 the sets $\{\mathcal{S}_k\}$ partition its index range. Regrouping the terms is therefore an identity. $\square$

In code (`model.py:93-110`), abridged:

```
# --- Tier 3: instrumentation. Read-only; not on the trained path.
# Same stock layer as forward() calls -- the only change is need_weights.
attn_out, alpha = self.attn(query, kv, kv, key_padding_mask=key_padding_mask,
                             need_weights=True, average_attn_weights=False)
alpha = alpha[:, :, 0, :] # (B, H, L); one query

# Re-derive the value projection: rows 2E: of the packed in_proj ARE W_V, b_V.
v = F.linear(kv, self.attn.in_proj_weight[2 * E:], self.attn.in_proj_bias[2 * E:])
v = v.view(B, L, H, hd).permute(0, 2, 1, 3) # (B, H, L, hd)

for name, sel in spans.items():
    o = torch.einsum('bhl,bhld->bhd', alpha[:, :, sel], v[:, :, sel])
    parts[name] = F.linear(o.reshape(B, E), self.attn.out_proj.weight)
return parts, attn_out # attn_out is the stock layer's own output
```

Three things to note. The `einsum` contracts over the selected positions $\mathcal{S}_k$ and the head dimension, and `F.linear` *without* a bias argument applies W_O to the concatenated heads, which is exactly $\sum_h W_O^{(h)}(\cdot)^{(h)}$; b_O is deliberately excluded and routed to the `bias` bucket at `model.py:332`. Masked positions carry $\alpha_j = 0$, so an absent modality’s peak bucket is exactly zero. And the function returns `attn_out` — the *stock* layer’s output — next to the hand-computed `parts`, so Proposition 2 is not an assumption about PyTorch’s internals but a claim that can be, and is, checked against them.

Remark (Why the value projection is recomputed). `nn.MultiheadAttention` returns the attention weights but not the per-position values, and the weighted sum over j happens inside the fused kernel. Splitting by source therefore requires re-deriving v outside the layer. Slicing rows `2d:3d` of `in_proj_weight` is not an approximation — that block *is* W_V , by the layer’s own packing convention (§6.1) — but it is a coupling to that convention, and it is the one place where the instrumentation reaches into stock internals rather than calling a documented interface.

Remark (Two caveats at this step). (i) *The weights α are softmax-coupled.* The quantity $p_{k,\ell}$ decomposes the output *given the realised attention pattern*. It is not the counterfactual “what if $\mathcal{S}_k$ were absent”: removing a span renormalises α over the survivors. This is the formal reason attribution and ablation are distinct measures.

(ii) *The value bias b_V is apportioned, not isolated.* Since $v_j^{(h)}$ includes $b_V^{(h)}$ and $\sum_j \alpha_j = 1$, the value bias contributes $\sum_h W_O^{(h)} b_V^{(h)}$ in total, divided among buckets in proportion to their attention mass. Unlike b_O it is not separated out. Being source-independent, it slightly inflates high-attention buckets. With the bias bucket at 0.041 of the budget against 0.72 for the feedforward term this is second-order, but it is a genuine approximation inside an otherwise exact split.

7 LayerNorm is linear in the components of its input

Tier 1 — stock PyTorch. `nn.LayerNorm` normalises each vector over its feature dimension and then applies a learned elementwise gain and bias: $\text{LN}_{\gamma,\beta}(z) = \gamma \odot (z - \mu(z)) / \sigma(z) + \beta$, with μ and σ the mean and standard deviation of z *itself*. Each `CrossAttentionBlock` owns two, `norm1`

after attention and `norm2` after the feedforward; γ and β are trained parameters. This section establishes a property of that stock layer; the tier-3 helper `_ln_component` then applies it.

Let $C = I_d - \frac{1}{d}\mathbf{1}\mathbf{1}^\top$ be the centering projector and

$$\sigma(z) = \sqrt{\frac{1}{d}\|Cz\|_2^2 + \varepsilon},$$

which is `_ln_sigma`; note `var(..., unbiased=False)` equals $\frac{1}{d}\|Cz\|^2$ exactly. Then $\text{LN}_{\gamma,\beta}(z) = \gamma \odot Cz / \sigma(z) + \beta$. For a *fixed* scalar $s > 0$ define the linear operator

$$A_{\gamma,s} := \frac{1}{s} \text{diag}(\gamma) C \in \mathbb{R}^{d \times d}. \quad (3)$$

Lemma 1 (Component transport). *For any decomposition $z = \sum_k c_k$,*

$$\text{LN}_{\gamma,\beta}(z) = \sum_k A_{\gamma,\sigma(z)} c_k + \beta.$$

Proof. $A_{\gamma,\sigma(z)}$ is linear, so $\sum_k A_{\gamma,\sigma(z)} c_k = A_{\gamma,\sigma(z)} \sum_k c_k = A_{\gamma,\sigma(z)} z = \gamma \odot Cz / \sigma(z)$. Adding β once gives $\text{LN}_{\gamma,\beta}(z)$. $\square$

The entire content of the lemma is that σ is evaluated on the *full* z and then held fixed. The only nonlinearity in LayerNorm is the scalar $\sigma(\cdot)$; freezing it makes the map exactly linear on that forward pass. This is why the implementation threads `sigma` in from outside rather than recomputing it per component:

```
# --- Tier 3: instrumentation helper.
def _ln_component(ln, c, sigma):
    # sigma must be computed from the full z, not from c
    return ln.weight * (c - c.mean(-1, keepdim=True)) / sigma
```

Remark. $A_{\gamma,\sigma(z)}$ is a *data-dependent* linear map. It is not the Jacobian of LN, and Lemma 1 does not linearise LN as a function — it is a statement about one realised forward pass. The requirement that β be added exactly once is also why `bias` must be its own bucket: mapping every component through LN naively would add β once per bucket.

8 The block recursion and transport operators

Block ℓ (`model.py:326-348`), with $q_0 = g$:

$$\begin{aligned} z_\ell^{(1)} &= q_\ell + a_\ell, & s_\ell^{(1)} &= \sigma(z_\ell^{(1)}), & u_\ell &= \text{LN}_\ell^{(1)}(z_\ell^{(1)}), \\ f_\ell &= \text{FFN}_\ell(u_\ell), \\ z_\ell^{(2)} &= u_\ell + f_\ell, & s_\ell^{(2)} &= \sigma(z_\ell^{(2)}), & q_{\ell+1} &= \text{LN}_\ell^{(2)}(z_\ell^{(2)}). \end{aligned}$$

Residual addition distributes over buckets trivially. Writing $A_\ell^{(i)} := A_{\gamma_\ell^{(i)}, s_\ell^{(i)}}$ as in (3), define the *block transport*

$$\Psi_\ell = A_\ell^{(2)} A_\ell^{(1)}, \quad \Psi_{>\ell} = \Psi_{N-1} \Psi_{N-2} \cdots \Psi_{\ell+1} \quad (\Psi_{>N-1} = I).$$

There are three injection points, hence three transports to the output:

Injected at	Passes through	Operator
g , before block 0	all blocks $0, \dots, N-1$	$T^{\text{init}} = \Psi_{N-1} \cdots \Psi_0$
attention of block ℓ	$A_\ell^{(1)}, A_\ell^{(2)}$, then blocks $> \ell$	$T_\ell^a = \Psi_{>\ell} \Psi_\ell$
feedforward of block ℓ	$A_\ell^{(2)}$, then blocks $> \ell$	$T_\ell^f = \Psi_{>\ell} A_\ell^{(2)}$

These operators are never materialised as matrices. The `stack` tensor of shape $(|\mathcal{K}|, B, 1, d)$ holds every bucket’s current value, and a single broadcast call to `_ln_component` advances all of them one step (`model.py:336, 346`). Buckets for blocks not yet reached hold zeros, and $A_{\gamma,s}\mathbf{0} = \mathbf{0}$, so preallocating the full set is safe.

Theorem 1 (Exact additive decomposition).

$$q_N = \underbrace{T^{\text{init}}g}_{\text{cls_init}} + \sum_{\ell=0}^{N-1} \sum_{k \in \mathcal{S}} \underbrace{T_\ell^a p_{k,\ell}}_{(\mathbf{k}, \ell)} + \sum_{\ell=0}^{N-1} \underbrace{T_\ell^f f_\ell}_{(\text{ffn}, \ell)} + \underbrace{\sum_{\ell=0}^{N-1} [T_\ell^a b_O^{(\ell)} + T_\ell^f \beta_\ell^{(1)} + \Psi_{>\ell} \beta_\ell^{(2)}]}_{\text{bias}}$$

Proof. Induction on ℓ . Proposition 2 splits a_ℓ into per-source terms plus b_O ; Lemma 1 pushes the accumulated sum through each LayerNorm while adding each β exactly once; residual additions distribute. Composing the per-block maps gives the stated transports. $\square$

The three transports of the bias term correspond exactly to the three `+=` sites and their placement relative to the `_ln_component` calls:

```
# --- Tier 3: inside forward_contributions.
stack[bias_i] += block.attn.out_proj.bias # before A1 -> gets T^a
... stack = _ln_component(block.norm1, stack, sigma1)
stack[bias_i] += block.norm1.bias # after A1 -> gets T^f
... stack = _ln_component(block.norm2, stack, sigma2)
stack[bias_i] += block.norm2.bias # after A2 -> gets Psi_{>l}
```

Moving any one of those lines across an `_ln_component` call silently breaks the identity of Theorem 1.

Bucket count.

$$|\mathcal{K}| = \underbrace{2}_{\text{init, bias}} + N \cdot \left(\underbrace{2|\mathcal{M}|}_{\text{spans}} + \underbrace{1}_{\text{ffn}} \right) = 2 + 16 \cdot 11 = 178.$$

The aggregate contributions sum over ℓ (`model.py:351-354`), collapsing 178 buckets to 13: five modalities, five `:token` buckets, plus `ffn`, `cls_init` and `bias`. The 176 per-layer entries recorded in `perlayer_seed1_n256.json` are the $N \cdot 11$ terms.

9 From vectors to a budget that sums to one

Theorem 1 yields 178 *vectors*. To obtain scalars, project onto the output direction $\hat{q}_N = q_N / \|q_N\|$ and normalise once more by $\|q_N\|$:

$$\boxed{\text{share}_k = \frac{\langle c_k, \hat{q}_N \rangle}{\|q_N\|} = \frac{\langle c_k, q_N \rangle}{\|q_N\|^2}} \quad (4)$$

Corollary 1. $\sum_{k \in \mathcal{K}} \text{share}_k = 1$.

Proof. $\sum_k \langle c_k, q_N \rangle / \|q_N\|^2 = \langle \sum_k c_k, q_N \rangle / \|q_N\|^2 = \|q_N\|^2 / \|q_N\|^2 = 1$, using Theorem 1 for the middle equality. $\square$

Two properties of (4) are worth stating explicitly.

- It is *signed*. A bucket with $\langle c_k, q_N \rangle < 0$ receives a negative share, meaning it pulls against the output direction.
- It is a *scalar projection*: $\text{share}_k \|q_N\|$ is the length of c_k ’s component along q_N , and the orthogonal component of c_k is discarded. A bucket may be large in norm and score approximately zero by being orthogonal to q_N . The double normalisation is what makes the quantity dimensionless and additive to one, rather than a cosine, which would not sum.

Numerical verification. Over $n = 23,648$ test molecules (seed 1), the implementation reports $\sum_k \text{share}_k = 1.0000000119$ and $\text{share}_{\text{cls_init}} = -3.06 \times 10^{-19}$. The first figure is the integration test for Propositions 1–2, Lemma 1 and Theorem 1 jointly: any error in the partition, in the threading of σ , or in the placement of the bias updates breaks it. The second says that after sixteen LayerNorms the learned initial query contributes nothing whatsoever to the final CLS; it functions purely as a seed for the first attention.

10 Results: the modality budget

All figures below are measured on the three `marina-final-run` seed checkpoints, $n = 23,648$ test molecules each, with all five modalities present (mean `all_inputs` cosine 0.9547).

Table 3: The measured budget. *Share* is (4) over the whole budget; *peaks-only* renormalises over the five modality buckets, and is the column to use for comparing modalities. *Mod-token* is the *m:token* bucket. The first four columns are means over three seeds; ablation drops are given per seed.

Modality	Share	Peaks-only	Mod-token	Live tokens	Ablation drop (s0 / s1 / s2)
HSQC	0.1038	0.4474	+0.0038	22.6	0.0130 / 0.0124 / 0.0127
¹³ C NMR	0.0589	0.2539	+0.0016	25.6	0.0056 / 0.0056 / 0.0058
MS	0.0393	0.1692	+0.0003	33.6	0.0054 / 0.0067 / 0.0067
¹ H NMR	0.0279	0.1204	+0.0016	25.3	0.0037 / 0.0034 / 0.0033
MW	0.0021	0.0092	+0.0021	1.0	0.0006 / 0.0007 / 0.0006

Three observations.

1. **The feedforward path dominates.** Only about 23% of the final CLS direction comes from modality tokens at all; 0.714 is written by the feedforward path and 0.045 by biases. This is why Table 3’s peaks-only column, not its share column, is the meaningful cross-modality comparison.

Table 4: The three buckets that are not attributable to a source. Together with the five modality and five mod-token buckets these complete the budget of Corollary 1.

Bucket	Seed 0	Seed 1	Seed 2	Mean
ffn	0.71217	0.71991	0.70859	0.71355
bias	0.04510	0.04064	0.04950	0.04508
cls_init	-6.7×10^{-21}	-3.1×10^{-19}	-3.5×10^{-25}	≈ 0

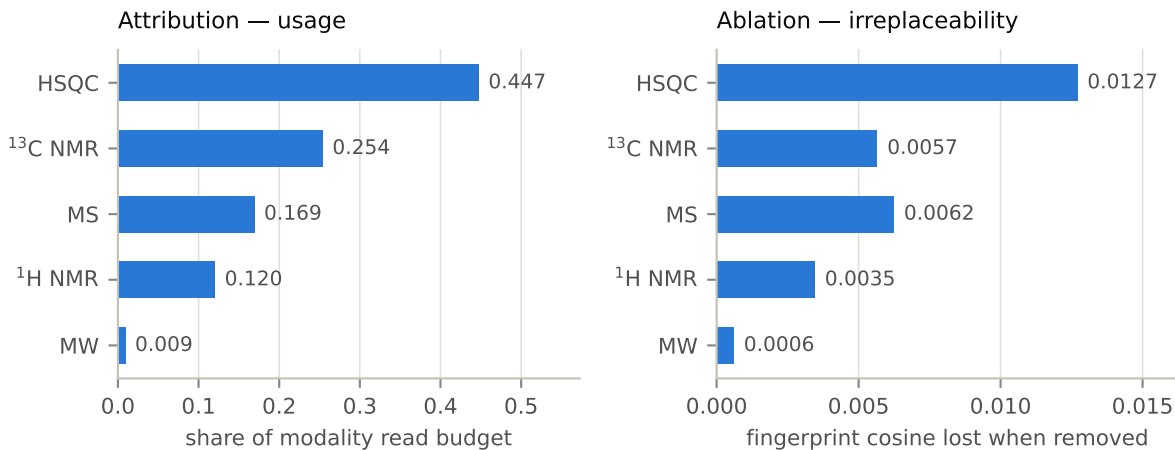


Figure 1: Usage against irreplaceability, mean over three seeds. The two measures are on different scales by roughly 30×, so they are shown as two panels rather than one chart with two axes; the category order is fixed by attribution in both. The rankings agree except for one adjacent swap (¹³C NMR and MS), and the magnitudes do not agree at all.

2. **HSQC dominates by both measures** — 45% of the peaks-only budget and the largest ablation drop — consistent with ME-HSQC being the strongest single modality in the SPECTRE results.
3. **The two measures agree on order and disagree on scale.** HSQC carries 45% of the read budget, yet removing it costs 0.013 cosine. Spearman’s ρ between the peaks-only share and the ablation drop is 1.0, 0.9, 0.9 across seeds 0–2; on seeds 1 and 2 the single adjacent swap is ^{13}C NMR against MS.

The disagreement in scale is not noise. MARINA trains with 50% dropout on every modality (`DROP_PERCENTAGE` in `src/modules/core/const.py`), so it is explicitly optimised to route around any single missing input. Ablation therefore measures irreplaceability *conditional on the other four modalities remaining*, and systematically understates any modality whose information is carried redundantly elsewhere. Attribution has no such conditioning — it reads off one forward pass with everything present. The gap between the columns is the redundancy signal, and §11 measures it directly.

Remark (`mw` and `cls_init`). Two buckets are effectively inert. `mw` takes 0.9% of the peaks-only budget and costs 0.0006 cosine to remove, and its mod-token share (+0.0021) equals its peak share (+0.0021) — half of what it contributes is the learned fact that it is present, not its value. Separately, `share_cls_init` is zero to machine precision in every seed: after sixteen LayerNorms the learned initial query survives not at all, and functions purely as a seed for the first attention.

11 Results: redundancy within the NMR block

If ablation understates redundant modalities, the direct test is to ablate the redundant group jointly. Define

$$S = \frac{\text{drop}(\{\text{hsqc}, \text{c_nmr}, \text{h_nmr}\})}{\sum_{m \in \{\text{hsqc}, \text{c_nmr}, \text{h_nmr}\}} \text{drop}(\{m\})}.$$

$S \approx 1$ means the members carry independent information and losses are additive. $S \gg 1$ means they are substitutes: each was covering for the others, so removing them one at a time measured almost nothing real.

Table 5: Joint versus individual ablation of the three NMR modalities, per seed. Spearman is between the peaks-only share and the ablation drop over the five modalities.

	Baseline cos	$\sum_m$ drop	Joint drop	S	Spearman ρ
Seed 0	0.9548	0.0223	0.2049	9.21	1.0
Seed 1	0.9546	0.0213	0.1763	8.27	0.9
Seed 2	0.9548	0.0219	0.1746	7.98	0.9
Mean	0.9547	0.0218	0.1853	8.49	0.93

Structurally this is an interaction-information or Shapley-synergy measure evaluated on the loss surface rather than on entropies. Its practical reading: **any one NMR modality can be reconstructed from the others; all three cannot.** It also bounds how much weight a single-modality ablation table can bear — including the dereplication tables — since every such number is measured with the redundant partners still present.

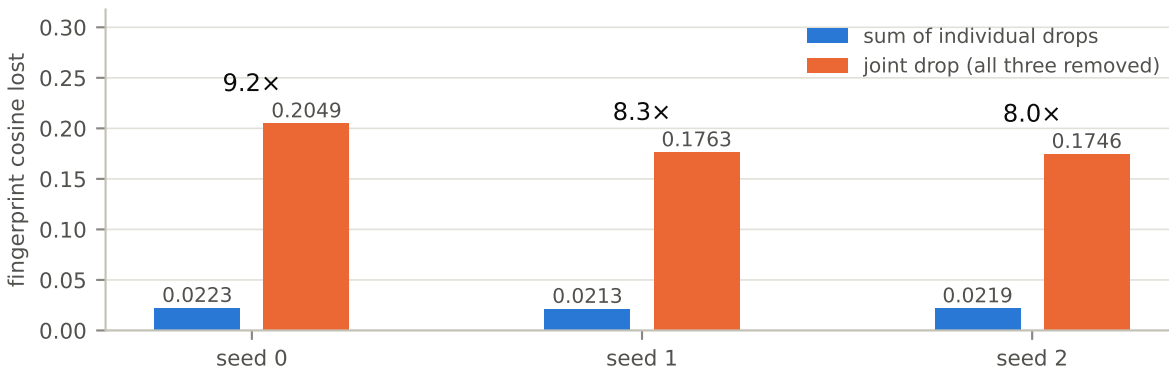


Figure 2: The NMR block is jointly essential and individually disposable. Removing HSQC, ^{13}C and ^1H *together* costs 8.0–9.2 $\times$ more fingerprint cosine than the sum of removing them one at a time.

12 The other redundancy: the fingerprint target

Every ablation number in §10–§11 is denominated in *fingerprint cosine*. That unit is not neutral, and a companion analysis measured exactly how far from neutral it is (Datasets/fp-redundancy/, MARINA1, checkpoint final1). Its findings belong here because they bound how the numbers above should be read — and because they are the same phenomenon at the other end of the model.

12.1 The target carries far less information than its width

EntropyFPLoader.setup selects the 16,384-bit fingerprint by ranking candidate circular substructures on *marginal* binary entropy. Binary entropy is monotonic in p below 0.5 and the selected bits are rare (median presence 0.0009, 471 of 518,901 molecules), so “top- K by entropy” is exactly “top- K by frequency”. Nothing in that criterion penalises bits for being correlated with each other.

They are. For any tree T over the bits, $H(\text{joint}) \leq \sum_i H_i - \sum_{(i,j) \in T} I(i;j)$, and the maximum spanning tree of the mutual-information graph makes the bound tightest:

Table 6: Realised capacity of the fingerprint. About 0.9% of nominal width is information — though still 7.7 $\times$ what is needed to index the retrieval set uniquely.

	Bits
Nominal width	16,384
$\sum_i H_i$ (marginal entropies)	422.6
– Chow–Liu MST mutual information	275.3
Joint entropy (tightest tree upper bound)	≤ 147.4
Needed to index the 519K retrieval set	19.0

The structure behind that number:

- 10,608 of 16,384 bits have some j with $P(i | j) = 1$. This is one-way implication — $P(i | j) = 1 \iff \text{supp}(j) \subseteq \text{supp}(i)$ — so j is a more specific bit that never fires without i .

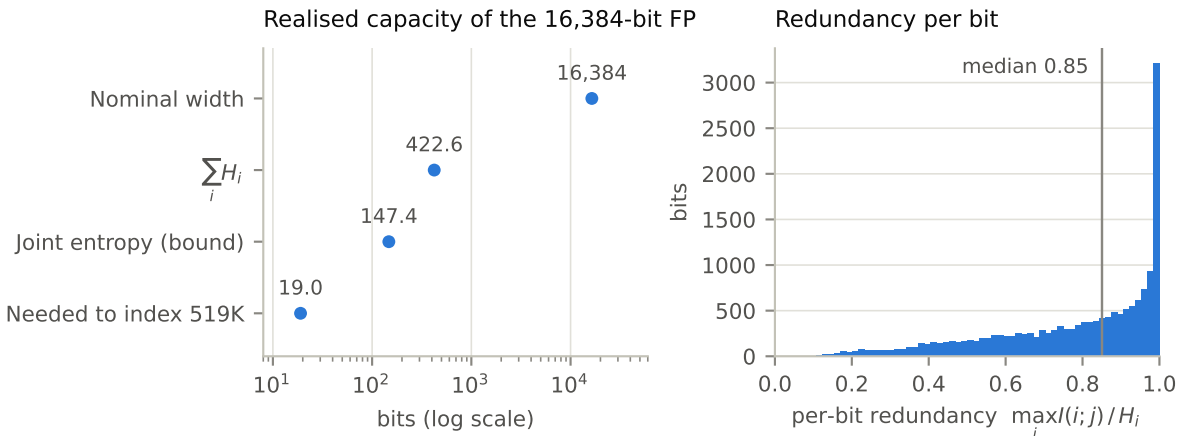


Figure 3: Left: the capacity ladder of Table 6, on a log axis — shown as points rather than bars, since bars on a log scale misstate proportion. Right: the per-bit redundancy $\max_j I(i; j) / H_i$ over all 16,384 bits. The mass piled against 1.0 is bits whose information is almost entirely duplicated elsewhere in the code.

- 1,695 bits are **exact duplicate columns**, collapsing into 592 groups.
- Median per-bit redundancy $\max_j I(i; j) / H_i = \mathbf{0.851}$; 6,731 bits exceed 0.9 and 2,799 exceed 0.99. This is not an artifact of rare bits — the median is still 0.686 in the *most frequent* decile.
- By RDKit substructure containment over the 2,808 exact-duplicate pairs, 65.7% are **radius nesting** (the ECFP family itself: radius r and $r + k$ around the same environment) and 34.3% are **congeneric series**. The largest duplicate group is 28 bits over 246 molecules — gangliosides differing only in ceramide acyl chain length, where 28 slots encode “this is a ganglioside”.

12.2 The redundancy buys nothing

The natural defence of a redundant code is that it is an error-correcting one: a model predicting bits from NMR gets several noisy shots at the same target. That defence fails. Within exact-duplicate groups — where the target column is *identical*, so the model sees g copies of one label — the mean pairwise correlation of prediction errors is $\rho = 0.973$, against 0.007 for frequency-matched control pairs. Averaging g such errors gives

$$\frac{\text{Var}}{\sigma^2} = \frac{1 + (g - 1)\rho}{g} = 0.982, \quad \text{versus } 0.349 \text{ if the errors were independent.}$$

There is essentially zero error correction. The duplicated bits act as a pure weight multiplier on their substructure family.

Nor can the waste be reclaimed by fixing the retrieval metric. Holding model and bits fixed and changing only the similarity to a weighted cosine $\text{sim}_w(q, r) = \langle s \odot q, s \odot r \rangle / (\|s \odot q\| \|s \odot r\|)$ with $s = \sqrt{w}$ gives Table 7: down-weighting redundant bits never helps, and past the exact-duplicate null it hurts monotonically in decorrelation strength.

Table 7: Down-weighting redundant bits at retrieval time, $n = 3,000$ test molecules, paired. Collapsing exact duplicates is a clean null; everything stronger degrades top-1 monotonically in decorrelation strength.

Scheme	w	Top-1	Δ	McNemar p
uniform (current)	1	0.7587	—	—
dup	1/exact-group-size	0.7597	+0.0010	0.55
soft0.99	1/#partners ≥ 0.99	0.7553	−0.0033	0.099
soft0.9	1/#partners ≥ 0.9	0.7513	−0.0073	0.0021
soft0.7	1/#partners ≥ 0.7	0.7473	−0.0113	0.0027
soft0.5	1/#partners ≥ 0.5	0.7307	−0.0280	6×10^{-9}
idf (control)	$\log(N/\text{count})$	0.7230	−0.0357	9×10^{-24}

Remark (What the sweep does and does not show). Reweighting at test time introduces a train/test mismatch independent of the redundancy question: the model was trained under a uniform-weight loss and its predictions are calibrated to the redundant code. That the `idf` control hurts about as much as `soft0.5` is consistent with mismatch, not redundancy, driving the degradation. The sweep therefore rules out a *free* post-hoc win; it does not show that a low-redundancy fingerprint would underperform. Settling that needs MI-aware bit *selection* and a retrain.

12.3 What this means for the numbers above

Three consequences for reading §10–§11.

1. **Absolute drops are in a distorted unit.** Cosine over this fingerprint implicitly weights each substructure family by how many correlated copies the frequency ranking happened to pick up — up to 28 for one ganglioside core. A drop of 0.013 cosine is therefore *not* “1.3% of the chemically relevant information”. It is 1.3% of a quantity whose weighting is an artifact of bit selection.
2. **Rankings are more robust than magnitudes.** Every reweighting short of the most aggressive moves top-1 by at most 0.011, so the *ordering* of the five modalities in Table 3 is unlikely to be a metric artifact — consistent with its agreement with attribution, which does not touch the fingerprint at all.
3. **The superadditivity ratio is comparatively insulated.** S is a ratio of two cosine drops, so a multiplicative distortion common to numerator and denominator partly cancels. This is a heuristic, not an invariance — the joint and individual ablations need not distort identically — but it makes $S \approx 8.5$ a safer number than the 0.0218 and 0.1853 it is built from.

The broader point is the symmetry. §11 finds redundancy on the *input* side: five modalities whose NMR block is 8.5× superadditive, because any one can be reconstructed from the others. This section finds redundancy on the *output* side: a 16,384-bit target realising ≤ 147.4 bits. MARINA has substantially fewer effective channels than nominal ones at both ends, and neither redundancy was designed in — the input one is a consequence of 50% modality dropout, the output one of ranking bits on marginal rather than joint entropy.

13 Results: refining the budget by depth

Theorem 1 indexes contributions by (k, ℓ) but the reported shares sum over ℓ . Restoring that index gives two per-block quantities.

Injection — the norm of what block ℓ wrote, at the moment of writing, before any later LayerNorm touches it:

$$\text{inj}(k, \ell) = \|p_{k, \ell}\|_2 \quad (\text{and } \|f_\ell\|_2 \text{ for } k = \text{ffn}).$$

Unsigned, in raw activation-norm units, with no cross-modality normalisation.

Survival — the same vector after transport to the output, projected onto the final direction exactly as in (4):

$$\text{sur}(k, \ell) = \frac{\langle T_\ell^a p_{k, \ell}, \hat{q}_N \rangle}{\|q_N\|}.$$

Signed, dimensionless, and a strict refinement rather than a new measure:

Corollary 2 (Depth refinement). $\sum_{\ell=0}^{N-1} \text{sur}(k, \ell) = \text{share}_k$ for every bucket k .

Proof. Immediate from Theorem 1 and the linearity of $\langle \cdot, \hat{q}_N \rangle$. $\square$

Retention — their ratio, $\text{ret}(k, \ell) = \text{sur}(k, \ell) / \text{inj}(k, \ell)$: what one unit of injected norm is worth at the output. Note it folds together two distinct effects — attenuation of magnitude by the repeated division by σ , and misalignment with $\hat{q}_N$ — and cannot separate them, since the numerator is a signed projection and the denominator an unsigned norm.

Table 8: Per-block profile, $n = 256$, seed 1. The last two columns verify Corollary 2 numerically: summing survival over the sixteen blocks reproduces the aggregate share of (4) to nine decimal places. (Shares here are for this $n = 256$ subsample, so they differ slightly from Table 3’s $n = 23,648$ figures.)

Bucket	Peak inj.	At $\ell = 15$		$\sum_\ell \text{sur}$	share_k	diff
	block (value)	inj.	sur.			
HSQC	9 (19.4)	8.3	0.0623	0.109428	0.109428	2.5×10^{-9}
^{13}C NMR	3 (46.2)	4.8	0.0315	0.058190	0.058190	2.9×10^{-10}
MS	4 (134.2)	4.0	0.0231	0.038983	0.038983	6.5×10^{-10}
^1H NMR	4 (62.2)	2.5	0.0153	0.029919	0.029919	6.4×10^{-10}
MW	5 (37.7)	0.5	0.0014	0.002144	0.002144	7.0×10^{-11}
ffn	2 (4816)	—	—	0.712767	0.712767	1.8×10^{-9}

The pattern in Figure 4 and Table 8 is consistent across every modality: **the block that reads hardest is never the block that matters**. MS injects 134.2 at block 4 — the loudest read anywhere in the model — and its survival there is below 10^{-4} . HSQC peaks at block 9 with 19.4 and survives 10^{-4} ; at block 15 it injects less than half as much, 8.3, and survives 0.0623, some $600\times$ more.

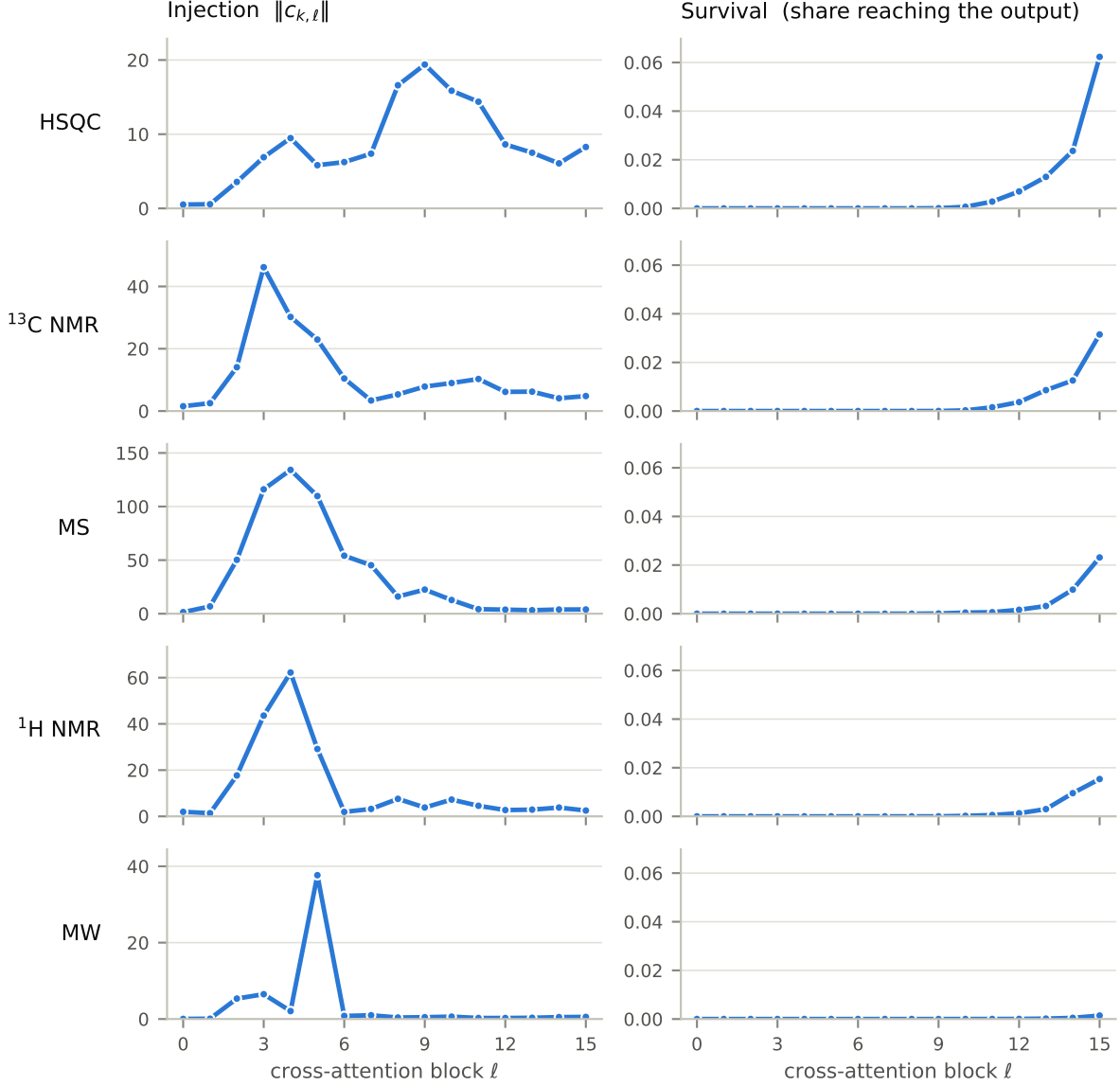


Figure 4: Where each modality is read, against where that reading survives. $n = 256$, seed 1. Survival shares one scale across rows because it is a share of one common budget; injection does not, because a raw activation norm carries no cross-modality meaning. **The columns peak at opposite ends of the stack.** Reading is heaviest at blocks 3–9; essentially none of it reaches the output, while block 15 alone — often a smaller write — delivers most of what survives.

The mechanism is the repeated $1/\sigma$ division of Lemma 1. Every bucket alive at block ℓ is rescaled again at every later block, and the feedforward writes that set σ early are enormous: `ffn` injects 4816 at block 2, two orders of magnitude above any modality’s read. Anything written before that is scaled toward nothing. Retention accordingly rises roughly geometrically over the last five blocks — for HSQC, $\times 10^3$: 0.2, 0.8, 1.7, 3.9, 7.5.

Remark (Why this is not yet a conclusion). Near-zero survival for blocks 0–9 is consistent with two incompatible stories: the early reads are discarded, *or* they matter through the feedforward path, whose bucket content depends on u_ℓ and hence on everything read at blocks $\leq \ell$. A linear decomposition cannot distinguish them, and the feedforward path holds 71% of the budget. That is what §14 is for.

14 Results: attention-depth suppression

A forward hook on each block’s `MultiheadAttention` suppresses reading in a contiguous set of blocks; the block still runs its feedforward and `LayerNorms`, it just reads nothing. No weights are modified. This is a causal intervention, so unlike §13 it follows the feedforward path.

Table 9: Cross-attention suppression, single checkpoint (`final1/best.ckpt`), $n = 512$, baseline cosine 0.9546. A dash means that k was not run for that direction.

k	First k blocks		Last k blocks	
	cos	drop	cos	drop
1	0.9546	0.0000	0.9178	0.0368
2	0.9546	0.0000	0.8413	0.1133
3	0.9546	0.0000	0.7611	0.1935
4	0.9546	0.0000	0.6826	0.2720
5	0.9547	−0.0001	—	—
6	0.9544	0.0002	0.5049	0.4498
8	0.9536	0.0010	0.2717	0.6829
10	0.9475	0.0072	—	—
12	0.7024	0.2523	—	—
14	0.4812	0.4734	—	—
16	0.1771	0.7775	—	—

Table 9 and Figure 5 settle the ambiguity left open in §13: the early reads are genuinely discarded, not routed through the feedforward path. Blocks 0–9 of a 16-block stack do nothing useful with their cross-attention in the converged model. Suppressing the first five is, if anything, very slightly *beneficial* (−0.0001, within noise) — exactly what one expects of a read that contributes nothing but numerical jitter.

The practical consequence is a prediction: an 8- or 12-layer model should match 16 layers at lower cost. Two caveats bound how far this result travels. It is a single checkpoint at $n = 512$, not seed-replicated, whereas the budget of §10 is three seeds at $n = 23,648$; and it used a local checkpoint copy rather than the three PVC `marina-final-run` checkpoints (baseline cosines agree, 0.9546 against 0.9548, but they are not the identical artifact).

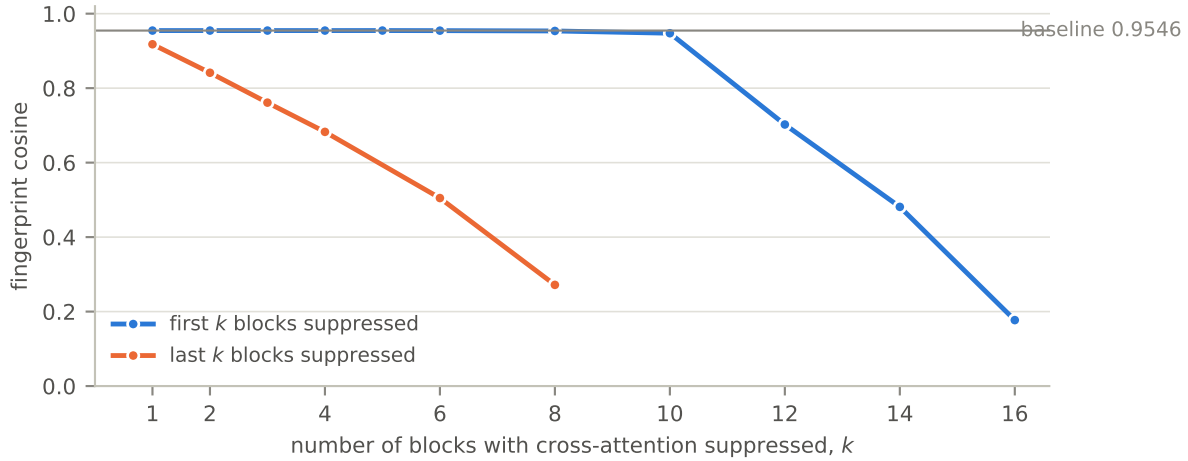


Figure 5: The asymmetry is the result. Suppressing cross-attention in the first ten blocks costs 0.0072 cosine; suppressing it in the last *one* costs 0.0368 — five times the damage from one block than from ten. The curve breaks sharply between $k = 10$ and $k = 12$.

15 Where exactness ends

Component	Status
span partition (Prop. 1)	exact
attention split by source (Prop. 2)	exact <i>given</i> α
value bias b_V	apportioned by attention mass, not isolated
LayerNorm transport (Lem. 1)	exact, σ frozen at its realised value
residual additions	exact
$b_O, \beta^{(1)}, \beta^{(2)}$	exact, in a separate bucket
feedforward	not decomposable — ReLU
α as a function of the input	not followed

The last two entries are the substantive limits. Since $f_\ell = W_2 \text{ReLU}(W_1 u_\ell + b_1) + b_2$ admits no exact source-wise split, the whole vector is assigned to the bucket (ffn, ℓ) — and that family of buckets absorbs 0.72 of the total budget. Crucially, the *content* of the feedforward bucket still depends on everything read at blocks $\leq \ell$, through u_ℓ . A modality whose attribution share is near zero may therefore still influence the output *via* the feedforward path, and this decomposition cannot detect it.

That is not a fixable defect of the accounting. It is the reason a separate, causal instrument (attention suppression, `early_attn_ablation.py`) is required. The decomposition tells you what the residual stream is *made of*; only intervention tells you what it *depends on*.